

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl



¿Qué es la Cardinalidad?

Definición

En el diseño de bases de datos, la **cardinalidad** define el número de entidades (filas) de una tabla que pueden asociarse con entidades de otra tabla a través de una relación.

- Determina las reglas de negocio en la estructura del software.
- Se expresa conceptualmente con números o letras (1:1, 1:N, M:N).
- Evita la redundancia de datos y garantiza la integridad referencial.

Las Llaves del Modelo: PK y FK

Para que exista una relación y se defina la cardinalidad, necesitamos:

- **Primary Key (PK) / Clave Primaria:** Identificador único de un registro en su propia tabla (ej: ID_Usuario). No se puede repetir.
- **Foreign Key (FK) / Clave Foránea:** Una columna en una tabla secundaria que hace referencia a la PK de la tabla principal. Permite la unión física de los datos.

Tipos de Cardinalidad

Existen tres tipos principales de relaciones que modelan la realidad:

- ① **Uno a Uno (1:1)**
- ② **Uno a Muchos (1:N)** o Muchos a Uno (N:1)
- ③ **Muchos a Muchos (M:N)**

A continuación, analizaremos cada una de ellas a nivel conceptual, su implementación técnica y cómo identificarlas correctamente.

1. Relación Uno a Uno (1:1)

Concepto

Un registro en la **Tabla A** se relaciona única y exclusivamente con un registro en la **Tabla B**, y viceversa.

- Es el tipo de relación menos común en bases de datos debido a que a menudo los datos podrían unirse en la misma tabla.
- Se utiliza por motivos de seguridad, rendimiento, o separación lógica de datos confidenciales.

Ejemplo Práctico de Relación 1:1

Piensa en la relación entre un **Empleado** y su **Expediente Médico**.

- Un empleado tiene solo un expediente médico dentro de la empresa.
- Un expediente médico pertenece exactamente a un solo empleado.

Tabla: Empleados	Tabla: Expedientes_Medicos
<u>ID_Empleado</u> (PK)	<u>ID_Expediente</u> (PK)
Nombre	Tipo_Sangre
Puesto	ID_Empleado (FK - Única)

Ejemplo 2 de Relación 1:1

Relación entre **Usuario** y **Credenciales de Acceso**:

- **Tabla Usuarios:** ID_Usuario (PK), Nombre, Correo.
- **Tabla Credenciales:** ID_Credencial (PK), Password_Hash, *ID_Usuario* (FK).

¿Por qué separarlo? Para que las consultas ordinarias de nombres y correos no carguen información crítica y encriptada de seguridad en memoria de manera innecesaria.

¿Cómo Identificar una Relación 1:1?

Para confirmar si estás ante una relación 1:1, hazte las siguientes preguntas bidireccionales:

Pregunta Bidireccional

- 1 ¿Puede un elemento de la Tabla A tener más de un elemento de la Tabla B? **(No)**
- 2 ¿Puede un elemento de la Tabla B pertenecer a más de un elemento de la Tabla A? **(No)**

Si ambas respuestas son un rotundo **NO**, la relación es de uno a uno (1:1).

2. Relación Uno a Muchos (1:N)

Concepto

Un registro en la **Tabla A** puede relacionarse con **cero, uno o muchos** registros en la **Tabla B**. Sin embargo, un registro en la **Tabla B** solo puede pertenecer a **un único** registro de la **Tabla A**.

- Es el pilar fundamental del diseño de bases de datos relacionales.
- Representa jerarquías, clasificaciones y pertenencia.

Ejemplo Práctico de Relación 1:N

Pensemos en una relación clásica: **Clientes** y **Facturas**.

- Un **Cliente** puede realizar muchas compras a lo largo del tiempo, generando **muchas facturas**.
- Una **Factura** específica es emitida a nombre de **un solo cliente** (no puede pertenecer a dos personas distintas al mismo tiempo).

Técnicamente, el **ID_Cliente** viaja a la tabla de Facturas como Clave Foránea (FK).

Estructura de Tablas en 1:N (Clientes y Facturas)

Tabla: Clientes (El "1")	Tabla: Facturas (El "N")
<u>ID_Cliente</u> (PK)	<u>ID_Factura</u> (PK)
Nombre_Cliente	Fecha
Pais	Monto
	ID_Cliente (FK)

- En la tabla Clientes, el ID 101 aparece solo una vez.
- En la tabla Facturas, el ID 101 puede aparecer repetido en múltiples filas representando cada compra de ese cliente.

Ejemplo 2 de Relación 1:N

Relación entre **Departamentos** y **Empleados**:

- Un departamento (ej: "Marketing") tiene asignados a **muchos** empleados.
- Cada empleado trabaja en **un único** departamento a la vez.

La clave primaria ID_Departamento se almacena como clave foránea en la tabla Empleados para saber a dónde pertenece cada trabajador.

¿Cómo Identificar una Relación 1:N?

Aplica el test bidireccional pensando en las reglas de tu negocio:

Test de Validación

- **De A hacia B:** ¿Un *Curso* puede tener múltiples *Estudiantes inscritos*? (**Sí**) → Aquí hay un extremo "Muchos".
- **De B hacia A:** ¿Un *Estudiante* pertenece a un único *Curso* en este sistema? Si la respuesta es **Sí**, entonces la relación es estrictamente 1:N.

3. Relación Muchos a Muchos (M:N)

Concepto

Múltiples registros en la **Tabla A** pueden asociarse con múltiples registros en la **Tabla B**, y viceversa.

- Los motores de bases de datos relacionales **no pueden implementar esta relación de forma directa**.
- Requiere obligatoriamente romper la relación usando una tercera tabla intermedia llamada **Tabla de Unión o Puente** (Junction Table).

Ejemplo Práctico de Relación M:N

Consideremos el caso de **Estudiantes** y **Materias**:

- Un **Estudiante** puede cursar **muchas materias** en el semestre (Cálculo, Historia, Programación).
- Una **Materia** (ej: Programación) tiene inscritos a **muchos estudiantes**.

No podemos poner la FK de materia en estudiantes, ni la FK de estudiante en materias de forma limpia.

Resolviendo el M:N con una Tabla Puente

Transformamos una relación **M:N** en dos relaciones **1:N** usando una tabla intermedia (ej: Inscripciones).

Tabla: Estudiantes (1)	Tabla Puente: Inscripciones (N)	Tabla: Materias (1)
<u>ID_Estudiante</u> (PK) Nombre	<i>ID_Estudiante</i> (FK) <i>ID_Materia</i> (FK) (Ambas forman una PK Compuesta)	<u>ID_Materia</u> (PK) Nombre_Materia

- De Estudiantes a Inscripciones la relación es **1:N**.
- De Materias a Inscripciones la relación es **1:N**.

Ejemplo 2 de Relación M:N

Relación entre **Productos** y **Órdenes de Compra (Pedidos)**:

- Un **Pedido** de un supermercado contiene **muchos productos** en el carrito.
- Un **Producto** (ej: Leche) aparece en **muchos pedidos** de clientes distintos.

La Tabla Puente: Se suele llamar *Detalle_Pedido*, y además de guardar las llaves foráneas, almacena atributos específicos de la relación como la *Cantidad* comprada y el *Precio Histórico*.

¿Cómo Identificar una Relación M:N?

El diagnóstico definitivo se da cuando al hacer la pregunta en ambas direcciones obtienes un "**Muchos**" como respuesta:

Regla de Oro para M:N

- ¿Un **Actor** puede participar en **muchas Películas**? (**Sí**)
- ¿Una **Película** puede tener a **muchos Actores** en su elenco? (**Sí**)

Si obtienes un Sí en ambos lados, estás ante un escenario **Muchos a Muchos (M:N)**.

Tabla Resumen de Identificación

Usa esta guía rápida para tus análisis de lógica de negocio:

Entidad A → Entidad B	Entidad B → Entidad A	Resultado
Máximo Uno	Máximo Uno	1:1
Máximo Muchos	Máximo Uno	1:N
Máximo Uno	Máximo Muchos	N:1 (Equivalente)
Máximo Muchos	Máximo Muchos	M:N

Sección II: Uniendo Tablas en Python con Pandas

¿Cómo implementamos las relaciones 1:N y M:N usando código en Pandas para nuestros análisis?

Implementando Relaciones 1:N con `pd.merge()`

Para simular el comportamiento de bases de datos relacionales en Python, utilizamos la función `merge`:

```
import pandas as pd

# Unir Clientes (1) con Facturas (N) usando la llave for nea
df_resultado = pd.merge(
    facturas_df,
    clientes_df,
    on='ID_Cliente',
    how='inner'
)
```

- **Equivalente SQL:** Realiza un *Inner Join* utilizando el ID compartido.
- **Ventaja:** Permite cruzar tablas relacionadas manteniendo intacta la integridad analítica de los datos.

Resolviendo Relaciones M:N con Pandas

Como las relaciones Muchos a Muchos requieren una tabla puente, en Pandas realizamos una cadena de cruces sucesivos (*chained merges*):

```
# 1. Unir Estudiantes con la Tabla Puente
df_temp = pd.merge(estudiantes_df, inscripciones_df,
                    on='ID_Estudiante')

# 2. Unir el resultado con la tabla de Materias
df_completo = pd.merge(df_temp, materias_df,
                       on='ID_Materia')
```

Optimización de Memoria y Tipos de Datos

En Python y Pandas, el uso eficiente de la memoria depende de los tipos de datos asignados:

```
# Convertir columnas categóricas para ahorrar memoria RAM
df['Pais_Origen'] = df['Pais_Origen'].astype('category')

# Eliminar columnas innecesarias antes de análisis pesados
df = df.drop(columns=['Comentario_Libre', 'Timestamp_Detalle'])
```

- Las columnas de tipo category reducen drásticamente el consumo de memoria en datasets grandes con baja cardinalidad.

Caso Práctico: Análisis de Redes Sociales en Python

Objetivo del Ejercicio

Escribe un script en Python que cargue las tablas de un archivo Excel, relacione la tabla de hechos con sus dimensiones y calcule métricas clave.

```
import pandas as pd

# Carga de datos tabulares
fact_interacciones = pd.read_excel('datos.xlsx', sheet_name='Fact_Interacciones')
dim_usuarios = pd.read_excel('datos.xlsx', sheet_name='Dim_Usuarios')

# Consolidación del modelo relacional
df_final = pd.merge(fact_interacciones, dim_usuarios, on='ID_Usuario')
print(df_final.groupby('Pais_Origen')['Likes'].sum())
```